

AURIGA
Linux, Git and ROS2 Sources
Fall 2026

- I. [What is ROS2?](#)
- II. [Getting Started With Linux](#)
- III. [Getting Started With Git & Github](#)
- IV. [ROS2 Docs](#)
- V. [ROS2 Basic Task](#)
- VI. [Submitting Task in Github](#)
- VII. [ROS2 Advanced Task](#)
- VIII. [ROS Advanced Task: Navigating Waiter Robot](#)

DEADLINE : 2 weeks

Suggested time intervals:

- | | |
|------------|--------|
| I, II, III | 2 Days |
| IV | 7 Days |
| V and VI | 5 Days |

What is ROS2?

ROS 2 is a software framework that provides a set of libraries, tools, packages, and standard conventions that make robot development easier, faster, and more modular.

At its simplest level, some of the most important tasks handled by ROS 2 include communication and message passing between different software units, even when they are written in different programming languages or use different communication protocols; defining robot models using standardized formats; and running different software units in parallel, among many other capabilities.

Of course, developing a robot without ROS 2 is completely possible. However, the tools and packages provided by ROS 2 can significantly improve both the development speed and the quality of the final system. For example, for a relatively advanced project like ATWORK, achieving the same level of functionality without ROS 2 could easily take many times longer; in my experience, potentially around ten times longer for this project.

Learning ROS 2 is highly recommended for anyone working in robotics. Beyond the technical tools it provides, ROS 2 connects you to a large and active robotics community where you can share your own projects and packages, while also exploring, testing, and building upon the work of others. This collaborative environment can accelerate your own development, benefit your team, and contribute to the rest of robotics community.

Getting Started With Linux

Linux is an operating system, just like Windows or macOS. The main difference is that Linux is open-source, highly customizable, and gives developers much more control over the system. It is also lightweight, stable, secure, and widely used for programming and technical applications. In robotics, this is especially important because we often need direct access to hardware, networking, sensors, and other system-level components. Linux also has a huge developer community and a large number of tools and libraries that make development easier.

We mainly use Ubuntu in robotics because it is one of the most widely supported Linux distributions for robotics development. The most important reason for us is ROS 2: ROS 2 officially supports specific Ubuntu versions, and most ROS 2 tutorials, packages, tools, and development environments are built and tested around Ubuntu. This makes setting up a robotics development environment much easier and reduces compatibility problems.

Follow these steps one by one:

- Install ubuntu22.04 as dual boot along side windows:

<https://youtu.be/mXyN1aJYefc?si=dldioEIuaPGDSFmW>

note1: Installing dual boot is fairly straight-forward but it is recommended to do it while in the lab so a senior member could help you out and we avoid any possible data loss which might happen even after you install ubuntu.

note2: I recommend you to not connect your ubuntu to wifi when you are installing it, because some of the driver updated may fail. After installation you're free to connect it to wifi and update anything you want.

- Learn Basics of Linux:

<https://youtu.be/oxuRxtr02Ag?si=iWsPHiesvfSnxUPH>

note: While you will learn everything you need along the way in the following months, it is recommended to watch this crash course to learn some of the basic concepts and commands of bash terminal

note : on Linux, authenticating your Github account is a bit different. To clone, Pull and Push you have two ways. One is SSH-Keys and the second one is PAT. tutorials for both is here:

<https://youtu.be/IuiH6cBtc58?si=gzvz0DuI8ImMFHMq>

<https://youtu.be/yVP3sYgd0bY?si=ZsnxmQd0-q5FKGmp>

- Learn more about Linux in the future (Optional):

<https://youtube.com/playlist?list=PL7ePwBdxM4nswZ62DvL58yJZ9W4-h0LLB&si=8hjqzXGka3MAXy9v>

<https://youtube.com/playlist?list=PL-tKrPVkKKE196tQGGunNbHVlBvq0RiaV&si=mo8cIEjCS6T0Xhcm>

note : these two courses are lpic1 and lpic2 from Jadi which are great sources to learn Linux. You can watch them in future in you found interest in Linux world. But is it not needed at the moment so skip them.

Getting Started With Git & Github

Git is a tool that helps us track and manage changes in a project. For example, when working on a robotics project, we can save different versions of our code, see what has changed, go back to an older version if something goes wrong, and work on different parts of the project without messing up the main code. GitHub is an online platform built around Git that allows us to store our projects online, share them with others, collaborate as a team, and use other people's projects and packages.

Follow these steps one by one:

- Create a Github account
- Learn Git & Github basics:

<https://youtu.be/yatvk0IqX84?si=nE6gCTaMl2ssa24d>

or

https://youtu.be/mAFoR0n0fHs?si=JHovGH6_h1sNJHdR

- Learn Git & Github on VScode:

<https://www.youtube.com/watch?v=twSxYaQikI&pp=ygUNZ2l0aHVlIHZzY29kZQ%3D%3D>

- Learn MARKDOWN format for documentation:

https://youtu.be/_PPWVRV6gbA?si=fgg4tZ5xCo4vaaiM

ROS2 Docs

Everything you need to know about basics concepts of ROS2 is available on the official ROS2 website so you do not need to watch any course, just pure document reading. The document is pretty easy to follow and you will get your hands on code every single time.

Note that:

- You need to have ubuntu 22.04 installed
- You need to have VScode installed
- You need to know some basic bash commands

And you are ready to start.

The website: <https://docs.ros.org/en/humble/>

Follow these steps one by one:

- Install “humble desktop version”:
<https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- Read basic concepts:
<https://docs.ros.org/en/humble/Concepts/Basic.html>
- Read every parts of Beginner : CLI tools
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html>
- Read every parts of Beginner : Client Libraries
<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html>
- From the Intermediate part, only read:
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-Main.html>

Note1: you can skip the parts that are not in your preferred language. If you code in C++, skip python and vice versa. Although launch files are only in python so read them anyways.

Note2: you can for now skip “Creating and using plugins (C++)” and “Using ros2doctor to identify issues”. You won’t need them at the moment.

Note3: these tutorials are really important. Spend enough time on each one of them. any logical or configuring issues/bugs you face, you must try to fix them no matter what. You should not skip any of them. we are also there to answer your questions.

Note4: As you learn ROS 2, you will come across many advanced topics related to configuration, compilation, external libraries, and other tools. In many cases, it is enough to understand what something does without knowing exactly how it works internally. Some of these topics are quite advanced, and even experienced developers may not know all the details behind them. So, if you find that understanding the internal workings of a package, CMake configuration, library, or similar tool is not directly related to our goals, you do not need to spend time learning it in depth. Just understand its purpose and how to use it, and, after discussing it with your mentors, feel free to skip the deeper details.

ROS2 Basic Task

Scenario: Waiter Service robot

- In a restaurant with a service robot as a waiter, there are 3 tables for customers
- Within a 5-second cycle, a new order comes from one of the tables. For 2 seconds, no order is coming so “robot’s state is free”. Customers spend 2 seconds choosing items with the robot by their side which “robot’s state is busy”. After that, the robot must receive the order (among available foods) in less than 1 second.
- The robot now tells the kitchen which food was ordered
- The kitchen receives the order from the robot and starts preparing the food. Each food item requires a different amount of preparation time.
- When the food is ready, the kitchen checks whether the robot is free or not. If the robot is available, the kitchen gives the ready-to-serve food to the robot, and the robot delivers it to the table. if the robot is not free, wait for the robot to be free.
- When a customer orders an item from the menu, the kitchen must first check whether that item is in stock. If the requested item is unavailable, it must be replaced with another available food item.
- When the restaurant runs out of all available food items, it closes, and no new orders should be accepted.

Menu:

	Item	Stock	Preparation time
1	Pizza	2	15 seconds
2	Burger	3	5 seconds
3	Pasta	3	7 seconds

Suggested setup:

3 Nodes customer_node, robot_node, kitchen_node
2 Topics /order, /robot_state
3 Services /food_available, /close_resturant, /food_ready
1 Launch file robot.launch.py

Keep in mind that:

- If you feel scenario is not complete or something is missing, it’s actually a way for you to practice the system design. Add anything you feel necessary for the system to work better.
- For simplicity, some actions don’t require extra messaging in the system. for example, robot delivering food to table can be a simple print in the robot node. Or the food availability doesn’t require robot as a man in the middle and the service between order node and kitchen node can be straight-forward. But if you design the system fully logical, that’s a positive point for you!
- If you have trouble choosing the message types for topics and services, consult the mentors, but remember that there is no right answer and you can design the system whatever you like. Also you can replace the suggested setup if you have better architecture in mind.

- Always check the edge cases and the race conditions of the system. For example, if you use an interrupt (blocking act) for pizza preparing time which is 15 seconds, you will miss the next orders from robot to kitchen.

Submitting Task in Github

- Fork This repository and clone the forked repo as you ROS workspace:
<https://github.com/AmirrezaRamesh/auriga-ros>
- Create packages and program the nodes, and commit each step of your progress with an understandable commit message
- You should fully document the result, important parts of the program, system design, rqt graph and the challenges you faced during the task in README.md file
- When the project is ready, open a “pull request” to my repository, then we review the code together and merge it, then run it on another machine

ROS Advanced Task: Navigating Waiter Robot

If, and only if you finish and submit the basic task within a week, you can go for the advance task with a one week deadline. before starting, watch these:

- Creating an action:
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Creating-an-Action.html>
- Writing an action server and client (C++):
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Writing-an-Action-Server-Client/Cpp.html>
- OccupancyGrid message:
https://docs.ros2.org/foxy/api/nav_msgs/msg/OccupancyGrid.html
- A* algorithm <https://www.youtube.com/watch?v=HR1TNa8Lp7w> <https://youtu.be/nbaSzCnmyec?si=bguWuLcYWEsEDenh>

Prerequisite: ROS Basic Task

DEADLINE : 1 week

Overview

In the basic task, the robot only exchanged messages. In this task, the robot must **actually navigate** to its goals on a grid map using the **A*** algorithm.

What you receive:

- A map file (20x20 grid).
- A map loader program, which publishes the map on /map (nav_msgs/msg/OccupancyGrid).

New changes:

- **Map and goals.** The tables and the kitchen each occupy only one cell, and going to a neighbor cell is considered as a reach. The goals are known and static. The obstacle positions can change for final testing, so your solution must not depend on a fixed obstacle layout.
- **Actions now require movement.** Every act that previously only exchanged a message must now make the robot actually navigate to those poses. For example, in a new 5-second cycle for orders, first choose which table is ordering, so the robot moves to that table. Or when the food is ready, the robot goes next to the kitchen to receive the food, then delivers it to the corresponding table.
- **Navigation.** The robot receives the goal and navigates to it using the “ros2 action” concepts (goal, feedback, result, cancel) and publishes /robot_pose. Path planning uses A*. The robot should move 1 cell/pixel per second using a timer (no blocking loops or sleep). Feel free to increase the food preparation and order cycle times if the navigation takes more time than they do.
- **Rest of the system.** Other parts of the system function the same (stock, substitutions, closing the restaurant, preparation times), although you might have to change some of

the designs because of the new tasks the system must handle, for example the robot being a shared resource between taking orders and delivering food.

- **Visualization node.** Visualize the navigation in a new node. Use `/map` and `/robot_pose` and print the state each second. No need to use any external tools like RViz or Qt.

Submission:

Same as the basic task: work in your fork, commit each step with a clear message, and open a pull request. In the `README.md`, document the system design, the rqt graph, a sample of the visualization output, and the challenges you faced.